

Solution Handover

AI Operations Platform — Multi-Tenant CRM, Contact-Centre & Sales Suite

Complete record of the solution and all development sessions

Period covered: 9 June 2026 – 22 June 2026

Document date: 22 June 2026

Repository: github.com/munirrazaa/AI-Operations-Platform- (branch: main)

Status: Delivered & tested · 24 database migrations · 21 release commits

Catalog

Contents.....	2
1. Executive Summary.....	3
2. What the Platform Is.....	3
3. Architecture & Technology.....	3
4. Multi-Tenancy & Security.....	4
5. Access-Control Model.....	4
5.1 Layer 1 — Entitlement (what is licensed).....	4
5.2 Layer 2 — Roles (who may create / edit / delete).....	4
5.3 Layer 3 — Record visibility (whose work you see).....	4
5.4 Separation of duties.....	4
6. Customer Onboarding.....	4
7. Operational Flow (Enquiry to Revenue).....	5
8. Development Timeline (All Sessions).....	5
9. Data Model — 24 Migrations.....	5
10. Current Status & Testing.....	5
11. Known Items & Roadmap.....	6
12. Access & Environment.....	6

1. Executive Summary

The AI Operations Platform is a multi-tenant Software-as-a-Service suite that lets a provider host many separate customer organisations ("tenants") on one system, each fully isolated from the others. Within each tenant it combines a CRM, a customer-service contact centre (including an AI voice agent), a sales-and-invoicing module, analytics, and team collaboration.

Between 9 and 22 June 2026 the platform progressed from an initial build to a tested, access-controlled solution. Headline outcomes:

- A complete CRM + contact-centre + sales suite with a licensing model that lets the provider sell each module and feature separately.
- A three-layer access-control model (what is licensed → who may act → whose records they see), plus a clean separation of duties for administrators.
- An end-to-end "enquiry-to-revenue" flow: a customer call becomes a service ticket or a sales deal, routed, worked, resolved, and confirmed.
- A hardened, tested codebase — 201 automated tests, 24 database migrations, and verified access rules.

2. What the Platform Is

Each customer organisation gets its own secure workspace. Staff log in and — depending on what the customer has purchased and the role they are given — use some or all of the following:

Capability	What it does
Core CRM	Contacts, companies, deals/pipeline and activities — the customer record.
Contact Centre	Tickets for complaints, enquiries and sales, with queues, routing, SLA/TAT timers, escalation and customer-satisfaction surveys.
AI Voice Agent ("Nadia")	A LiveKit-based voice bot that can take calls, log them and raise tickets automatically.
Sales & Invoicing	Invoices, billing contacts, payments, templates, a sales dashboard and settings.
Email Inbox	Shared team inbox with sending via the customer's own provider (SMTP/SendGrid/Microsoft 365).
Analytics & Reports	Dashboards and team reports, scoped to each viewer's remit.
Team Messaging	Internal staff chat.
Integrations	Connectors, webhooks and API keys.

3. Architecture & Technology

The system is a single code repository ("monorepo") split into shared packages and two runnable applications.

Layer	Technology
Frontend (web app)	React, TanStack Query & Table, Tailwind-style UI, Lucide icons, dnd-kit, LiveKit client.
Backend (API)	Node.js with Fastify; GraphQL via Mercurius; Zod validation; JWT auth; rate-limiting & security headers.
Database	PostgreSQL with Row-Level Security (per-tenant isolation); 24 ordered migrations.

Voice	LiveKit server SDK; the "Nadia" agent for inbound/outbound calls.
Email	Nodemailer (SMTP), SendGrid and Microsoft 365 Graph providers.
Documents	docxtemplater + PizZip for invoice/template generation.
Hosting	API on a VPS; frontend on Vercel; storage on S3-compatible backend; Redis optional.

4. Multi-Tenancy & Security

- Every record carries a tenant identifier and PostgreSQL Row-Level Security enforces that one tenant can never read another's data — at the database layer, not just in code.
- Authentication uses signed JWT tokens; a revocation list invalidates tokens on logout or password change.
- API keys (for integrations) are hashed, scoped and rate-limited.
- Security headers, CORS allow-listing and request validation are applied globally.
- Secrets (e.g. the email provider key) are kept in local environment files that are excluded from version control.

5. Access-Control Model

Access is governed by three independent layers plus a separation-of-duties rule. Together they answer: what did the customer buy, who may act, and whose records can they see.

5.1 Layer 1 — Entitlement (what is licensed)

The provider's super-admin licenses modules and the specific features within them when creating a workspace. This is the commercial ceiling, stored against the tenant. Unlicensed features are hidden from the menu and refused by the API.

5.2 Layer 2 — Roles (who may create / edit / delete)

Within the licensed set, the tenant administrator builds roles (Admin, Manager, Agent, Viewer and custom roles) and decides who may create, edit or delete each type of record. Four standard roles are auto-created with sensible defaults.

5.3 Layer 3 — Record visibility (whose work you see)

Visibility follows the line-manager reporting tree, applied as a hard filter: an agent sees only their own records; a line manager sees their team; a department manager sees their whole department. This governs Contacts, Deals, Activities, dashboards and pipeline totals consistently.

5.4 Separation of duties

The tenant administrator is a back-office role only — it manages users, roles, settings and integrations but cannot see operations or billing. Billing (both the subscription and customer invoicing) belongs to a Finance/Sales role, never the administrator.

6. Customer Onboarding

1. Super-admin creates the workspace in two steps: organisation & administrator details, then the licensed modules & features agreed with the customer.
2. The first tenant administrator is provisioned automatically with a one-time temporary password.
3. The temporary password is e-mailed to the customer (SendGrid); the super-admin can reset and re-send it at any time.

4. Four standard roles are auto-seeded; the tenant administrator then tailors roles and invites staff, setting each person's line manager and department.

7. Operational Flow (Enquiry to Revenue)

5. A customer contacts the support function — by phone (incl. the Nadia voice agent), email or web.
6. A ticket is raised and typed as a complaint, enquiry or sales opportunity.
7. Smart, capacity-aware routing sends it to the right queue/agent; the agent accepts it and the SLA/TAT timer starts.
8. The agent works the ticket through its stages (accepted → in progress → pending → resolved → closed), with milestones and escalation.
9. For a sales ticket, acceptance (or a manual "Convert to Deal" button) creates a linked deal in the sales pipeline so the opportunity is forecast — nothing is lost between teams.
10. On resolution the customer receives a satisfaction (CSAT) survey; the ticket and its live status also appear on the customer's CRM record.

8. Development Timeline (All Sessions)

Chronological record of the work, by date.

Date	Work delivered
9 Jun 2026	Initial platform build; LiveKit "Nadia" complaint ingestion; tsx API runtime.
10 Jun 2026	SQA remediation — 11 bugs fixed, 201/201 tests passing; Supabase TLS; Redis made optional; "Call Nadia" web-call (backend dispatch + token + widget).
11 Jun 2026	Automated SQA test suite added (201 tests).
13 Jun 2026	Milestone templates & queue-routing columns; deployment to new VPS.
19 Jun 2026	Fifteen gaps resolved — webhooks, S3 storage, email, scaling, analytics, mobile; migrations 012–017.
20–22 Jun 2026	Licensing/entitlement system; Sales & Invoicing; team messaging; sales-dashboard & invoice fixes; super-admin console clean-up.
22 Jun 2026	Separation of duties (admin-only tenant admin); hierarchy record-visibility; sales-ticket→deal conversion + button; SendGrid onboarding & password reset; TAT on customer record; Departments page fix; this handover.

9. Data Model — 24 Migrations

The database evolved through 24 ordered migrations, grouped below.

Group	Migrations	Coverage
Foundation	001–009	Core schema, billing, ticketing, email, voice bot, password reset, complaint fields, sales invoicing, Row-Level Security, roles/sector/permissions.
Enhancements	010–017	Gap remediation, milestone templates & routing, comment threading, line-manager links, system-role uniqueness, platform roles, permission normalisation, platform invoices, invoice templates.
Scale & Structure	018–024	Departments & opportunities, webhook queue, analytics materialised views, backup & storage, team messages, tenant entitlements, ticket→deal link.

10. Current Status & Testing

Area	Status
Automated test suite	201 / 201 passing
Tenant isolation (RLS)	Enforced & verified
Licensing / entitlement enforcement	Verified — unlicensed features return "not licensed"
Separation of duties (admin lockout)	Verified — operational & billing routes blocked for admin
Record visibility by hierarchy	Verified at agent / line-manager / manager levels
Sales ticket → deal conversion	Verified — auto + manual, idempotent
Departments management	Fixed & verified (was failing)
Onboarding email & password reset	Verified — live e-mail delivered via SendGrid

11. Known Items & Roadmap

- Optional: appoint a single company-wide operational manager (already supported by simply setting reporting lines — no code change needed).
- Optional: extend granular per-action permission checks (create/edit/delete keys exist in Roles and can be wired to more endpoints).
- Optional: enrich the analytics dashboards and team reports as usage grows.
- Data linkage note: staff are linked to departments by department name/type; a future migration could add a formal department identifier if richer department features are needed.

12. Access & Environment

Operational reference for the team. Secrets are NOT included in this document and are never committed to version control.

- Repository: github.com/munirrazaa/AI-Operations-Platform (branch main).
- Frontend: Vercel · API: VPS · Database: PostgreSQL (Supabase pooler, TLS) · Storage: S3-compatible · Redis: optional.
- Email: configured per environment via SendGrid / SMTP / Microsoft 365 keys held in local environment files only.
- Super-admin and tenant credentials are distributed separately through secure channels, not in this document.

13. Update — 2026-07-20

Live testing and hardening pass across every tenant, plus a real reliability fix found along the way.

- Sector-specific custom fields backfilled across 7 of 8 tenants — only Haier Electronics previously had its sector's full field set; the other tenants were missing most company/deal/ticket fields. 183 fields inserted, now verified matching across all tenants.
- Built full Sales/Support/Complaints + Field Sales department staff across all 8 tenants (previously only the demo tenant had this), wired into ticket routing queues, and live-tested end-to-end: tickets route to the correct department's queue, the right agent can see and accept them, wrong-department agents correctly cannot.

- Fixed a real reliability bug where a transient database network hiccup was crashing the entire API server instead of just the failing request — added proper error handling so the server now survives instead.
- Reset the universal test password for all 75 users across all 8 tenants (see Credentials file) — the previous one had had issues.
- Enforced the Sub-Admin Roles permission system server-side (previously only enforced in the frontend) across all ~65 Super Admin API routes.

14. Update — 2026-07-20 (Push 2): Critical Security Fix

A user-reported issue with the Voice Bot uncovered a serious cross-tenant data leak, fixed the same day.

- Every tenant's Voice Bot, across every sector, was falling back to a hardcoded default that was actually another real client's (HBL Microfinance Bank) confidential complaint-handling script. Discovered when Haier Electronics' bot introduced itself as HBL Microfinance Bank instead of Haier.
- Fixed at the code level: removed the hardcoded fallback entirely. Any tenant without its own configured script now safely falls back to a generic, brand-neutral script with no company-specific claims.
- Fixed Haier's own underlying data gap so it now correctly represents itself using its assigned Electronics Retail script.
- Separately fixed: tenants with zero allocated Voice Bot minutes were still able to take calls with no limit. Now, no allocated minutes means every call is routed straight to a human with a ticket already raised — enforced identically for every tenant in every sector.
- Verified via direct simulation and live API tests before this fix was shipped.

15. Update — 2026-07-21: Monorepo Build Reliability Fix

An internal safety-net fix — the platform's TypeScript type-checker had never been run cleanly across most of the backend, since production runs via a tool that skips type-checking entirely. Fixed in a controlled, isolated way; two small real bugs were found and fixed along the way.

- Fixed the full TypeScript build across the API server, core services, and all 11 backend modules — previously none of these had ever been individually type-checked, so real mistakes could have hidden undetected.
- Done safely: created a separate branch, took a safety backup (git tag + full solution zip) before starting, tested thoroughly, and only merged into the main codebase once fully verified — no risk taken with the live, working solution.
- Found and fixed two small real bugs while doing this: emails sent from the platform weren't correctly recording which staff member sent them, and one internal notification was silently failing to fire — both now fixed.
- Confirmed (separately, not yet fixed) that the live-call-audio-streaming feature has never actually worked, because a required plugin was never installed — logged as its own follow-up item, not part of this fix.
- No changes to the database, no changes to what any user sees or does — this was entirely a behind-the-scenes reliability and code-quality fix.